

Unix Process API

Learning Objectives

- Be able to use Unix process creation system calls.
- Be able to articulate how process isolation affects the behavior of parent and child processes.

Complementary Reading [*Operating Systems: Three Easy Pieces \(OSTEP\)* by Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, chapter 5: Process API](#)

Review Illusions/Reality activity from last week

- See canvas for the canvas quiz.
- See Monday lecture slides for answers and explanation.

`getpid()` [Process Identifiers]

- Unix systems provide each executing process with a unique identifying integer, called a **process ID or PID**.
- Note: PIDs can be reused after a process terminates.
- You can execute `getpid()` in a program; when executed it returns the PID of the currently executing process.
- Command line tools, such as `ps`, `top`, and `htop`, report processes by PID.
- See [lectures/unix/getpid_demo.c](#) for an example of using `getpid()`

`fork()` [Process Creation]

- `int fork()` is a system call that lets a program create a new process.
 - “Called once and returns twice”
 - We call the parent that calls `fork()` the **parent**
 - We call the process that is created the **child**
 - Memory between the parent and child is copied rather than shared.
- `fork()` Return value:
 - In the child process, `fork()` returns `0`
 - In the parent process, `fork()` returns the child’s PID
 - This difference is how we can tell which process is which in our programs!
- Unix systems provides `getppid()` to determine a process’ parent’s PID
- `init`: The original process
 - Unix systems create a single “original” process during boot

- Typically `systemd` on modern systems
 - Conventionally has PID of 1.
- See `lectures/unix/fork_demo.c` for an example program

`wait()` [Process Synchronization and Termination]

- What about when a process terminates?
 - Should we alert the parent?
 - Should we tell the parent about the process' return value?
 - A process that has terminated but not coordinated with its parent a **zombie process**
 - See `lectures/unix/zombie_demo.c` for an example
- When a parent coordinates with its child, we say that it **reaps** the child.
- `int wait()` is a system call that lets a process get the return value of its child
 - The system call blocks until its next child finishes executing and then returns.
 - The system call returns the return value of the child process.
 - There is a family of functions that have similar semantics (e.g., `waitpid`) with slightly different options.
 - See `lectures/unix/wait_demo.c` for an example
 - Be sure to use `htop` or `top` while running!
- What happens if a parent exits before the child is done?
 - Unix systems are designed so that `init` "adopts" the child
 - See `lectures/unix/adopt_demo.c` for an example.

`exec()` [Running new Programs]

- `int exec` is a system call that allows a process to run a new program
 - `exec` *only returns* if there is an error!
 - `exec` replaces the address space of the calling process with a new address space
 - `exec` does not replace the PID (or parent) of the child process
 - There are a whole family of functions that behave similarly to `exec`, such as `execv` and `exec1`.
 - See `lectures/unix/exec_demo.c` for an example
- The typical usage is for programs to call `fork` and then call `exec` in the child.